

TP1 – Deep Learning

SLURM, environnement Python, premier réseau de neurones et TensorBoard

Maroua Zoubir
Télécom SudParis – MAIA

21 septembre 2026

Table des matières

1	Utilisation de SLURM	2
2	Création d'un environnement virtuel Python	3
3	Exercices théoriques	4
3.1	Architecture et paramètres	5
3.2	Graphe de calcul et rétropropagation	6
3.3	Mise à jour des poids ET Association sortie / fonction de perte	6
4	Premier réseau de neurones sur CIFAR-10	7
5	Utilisation de TensorBoard	8

1 Utilisation de SLURM

Question. Une fois connecté au nœud de calcul, exécutez `nvidia-smi`. Quel est le modèle exact du GPU qui vous a été alloué ?

```
This cluster is reserved for teaching purposes only.

---

Bienvenue sur le cluster HPC Albator
Opéré par la Direction de l'Enseignement – Télécom SudParis

Commandes utiles :
  squeue          Lister les jobs en cours et en attente
  squeue -u $USER Lister vos propres jobs
  sbatch script.sh Soumettre un job
  scancel <jobid> Annuler un job
  sinfo           Afficher l'état des nœuds du cluster
  sacct           Afficher l'historique de vos jobs

Interface web : https://de.telecom-sudparis.eu/albator/
Support :      arthur.jovart@telecom-sudparis.eu

Ce cluster est réservé à l'enseignement.

#####
[controller] mzoubir@arcadia-slurm-controller:~
$ srun --partition=gpu --gres=gpu:1 --time=01:00:00 --cpus-per-task=1 --mem=8G --pty bash
[compute] [job 1532] mzoubir@starfighter-slurm-node-01-1:~
$ nvidia-smi
Wed Sep 16 11:54:58 2026
+-----+
| NVIDIA-SMI 595.84                Driver Version: 595.84          CUDA Version: 13.2         |
+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
| 0    NVIDIA L4              Off          | 00000000:06:10.0 Off |            0         |
| N/A   34C    P8             16W / 72W   | 3MiB / 23034MiB |    0%      Default   |
|=====+=====+
+-----+-----+
| Processes:                        |
| GPU   GI    CI          PID    Type   Process name          GPU Memory |
| ID    ID    ID             |                 |           Usage      |
|=====+=====+
| No running processes found      |
+-----+-----+
[compute] [job 1532] mzoubir@starfighter-slurm-node-01-1:~
$
```

FIGURE 1 – Sortie de Nvidia-smi

Modèle du GPU alloué : le GPU alloué est un **NVIDIA L4**, avec environ **23 Go de mémoire** (23034 MiB), sur le nœud

Question. Notez la commande exacte que vous avez tapée pour annuler votre job.

```
[controller] mzoubir@arcadia-slurm-controller:~$ srun --account=csc_8607 --qos=interactive --job-name=tp1-interactif --partition=gpu --gres=gpu:1 --time=01:00:00 --cpu-s-per-task=1 --mem=8G --pty bash
[compute] [job 2391] mzoubir@starfighter-slurm-node-01-1:~$ queue -u $USER
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
2391 gpu tp1-inte mzoubir R 0:21 1 starfighter-slurm-node-01-1
[compute] [job 2391] mzoubir@starfighter-slurm-node-01-1:~$ scancel 2391
srun: Job step aborted: Waiting up to 32 seconds for job step to finish.
[2026-09-20T07:54:04.475] error: *** STEP 2391.0 ON starfighter-slurm-node-01-1 CANCELLED AT 2026-09-20T07:54:04 DUE to SIGNAL Terminated ***
```

FIGURE 2 – Annulation du job interactif

Question. Lisez le fichier de log généré dans le dossier `logs/`. Quel est le nom exact de ce fichier ?

```
[controller] mzoubir@arcadia-slurm-controller:~$ ls logs/
hello-slurm-1538.err hello-slurm-1538.out hello-slurm-4088.err hello-slurm-4088.out
[controller] mzoubir@arcadia-slurm-controller:~$
```

FIGURE 3 – Fichier de logs

```
[controller] mzoubir@arcadia-slurm-controller:~/logs$ cat hello-slurm-1538.out
Job 1538 on starfighter-slurm-node-03-1
Wed Sep 16 12:10:28 2026

+-----+
| NVIDIA-SMI 595.84                Driver Version: 595.84                CUDA Version: 13.2                |
+-----+-----+-----+-----+-----+-----+
| GPU  Name      Persistence-M | Bus-Id  Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf   Pwr:Usage/Cap |          Memory-Usage | GPU-Util  Compute M. |
|====+=====+====+=====+=====+=====+
|  0  NVIDIA L4      Off      | 00000000:06:10.0 Off |          0          |
| N/A   32C    P8      15W / 72W |  0MiB / 23034MiB |    0%    Default  |
+-----+-----+-----+-----+-----+-----+
+-----+
| Processes: |
| GPU   GI    CI          PID    Type   Process name                  GPU Memory |
| ID     ID     |                     |            |                     |      Usage |
+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+

Bonjour depuis SLURM !
```

FIGURE 4 – Fichier de logs

Le nom suit le motif `logs/%x-%j.out` défini dans le script, où `%x` est le nom du job (`hello-slurm`) et `%j` son JobID. Nom exact obtenu : **hello-slurm-1538.out**

Question. Expliquez brièvement avec vos propres mots la différence entre *ReqMem* et *MaxRSS*.

ReqMem est la mémoire que l'on a demandée à Slurm (ici 8 Go) : c'est la quantité réservée pour le job, qu'il l'utilise ou non. *MaxRSS* est la mémoire réellement consommée : le pic de mémoire résidente atteint par le job pendant son exécution. Comparer les deux permet d'ajuster la demande, pour ne pas réserver inutilement des ressources partagées.

2 Création d'un environnement virtuel Python

Question. Quelle commande utilisez-vous pour vérifier la version exacte de Python installée dans votre environnement actif, ainsi que le chemin du binaire utilisé ? Notez la commande et son résultat.

```

[compute] [job 2392] mzoubir@starfighter-slurm-node-01-1:~
$ python --version
Python 3.10.21
[compute] [job 2392] mzoubir@starfighter-slurm-node-01-1:~
$ which python
/mnt/hdd/homes/mzoubir/miniforge3/envs/deeplearning/bin/python
[compute] [job 2392] mzoubir@starfighter-slurm-node-01-1:~
$

```

FIGURE 5 – Version du python et le chemin de binaire de python

Question. Copiez-collez la sortie du script `check_gpu.py`. Si `CUDA available` retourne `False`, listez deux raisons possibles.

```

[compute] [job 2068] mzoubir@starfighter-slurm-node-11-1:~
$ python check_gpu.py
PyTorch version: 2.14.0+cu130
CUDA available: True
Device count: 1
Device 0 name: NVIDIA L4

```

FIGURE 6 – Sortie de `check-gpu`

Si `CUDA available` retournerait `False`, deux raisons possibles seraient :

1. aucun GPU n'a été alloué à la session (login node utilisé, ou option `-gres=gpu:1` oubliée dans le `srun`);
2. PyTorch a été installé dans une version CPU-only, ou sa version de CUDA est incompatible avec le pilote NVIDIA de la machine.

Question. Bonus : quelle commande permet d'afficher la version de `TensorBoard` que vous venez d'installer ?

```

[compute] [job 2392] mzoubir@starfighter-slurm-node-01-1:~
$ tensorboard --version
/mnt/hdd/homes/mzoubir/miniforge3/envs/deeplearning/lib/python3.10/site-packages/tensorboard/default.py:30: UserWarning:
pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources p
ackage is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  import pkg_resources
TensorFlow installation not found - running with reduced feature set.
2.20.0
[compute] [job 2392] mzoubir@starfighter-slurm-node-01-1:~
$

```

FIGURE 7 – version `TensorBoard`

3 Exercices théoriques

Les réponses de cette partie ont été rédigées à la main; les pages correspondantes sont insérées ci-dessous.

3.1 Architecture et paramètres

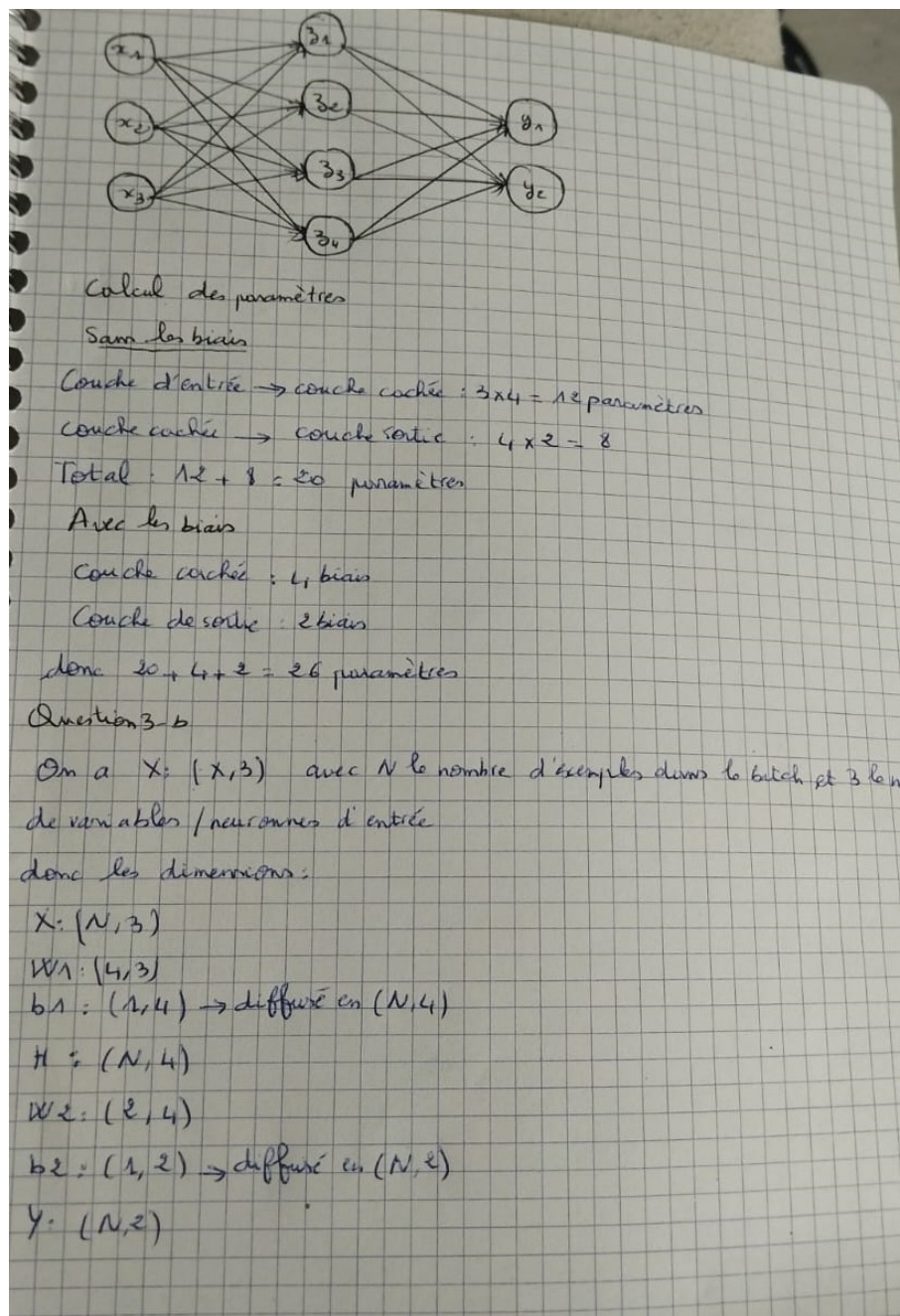
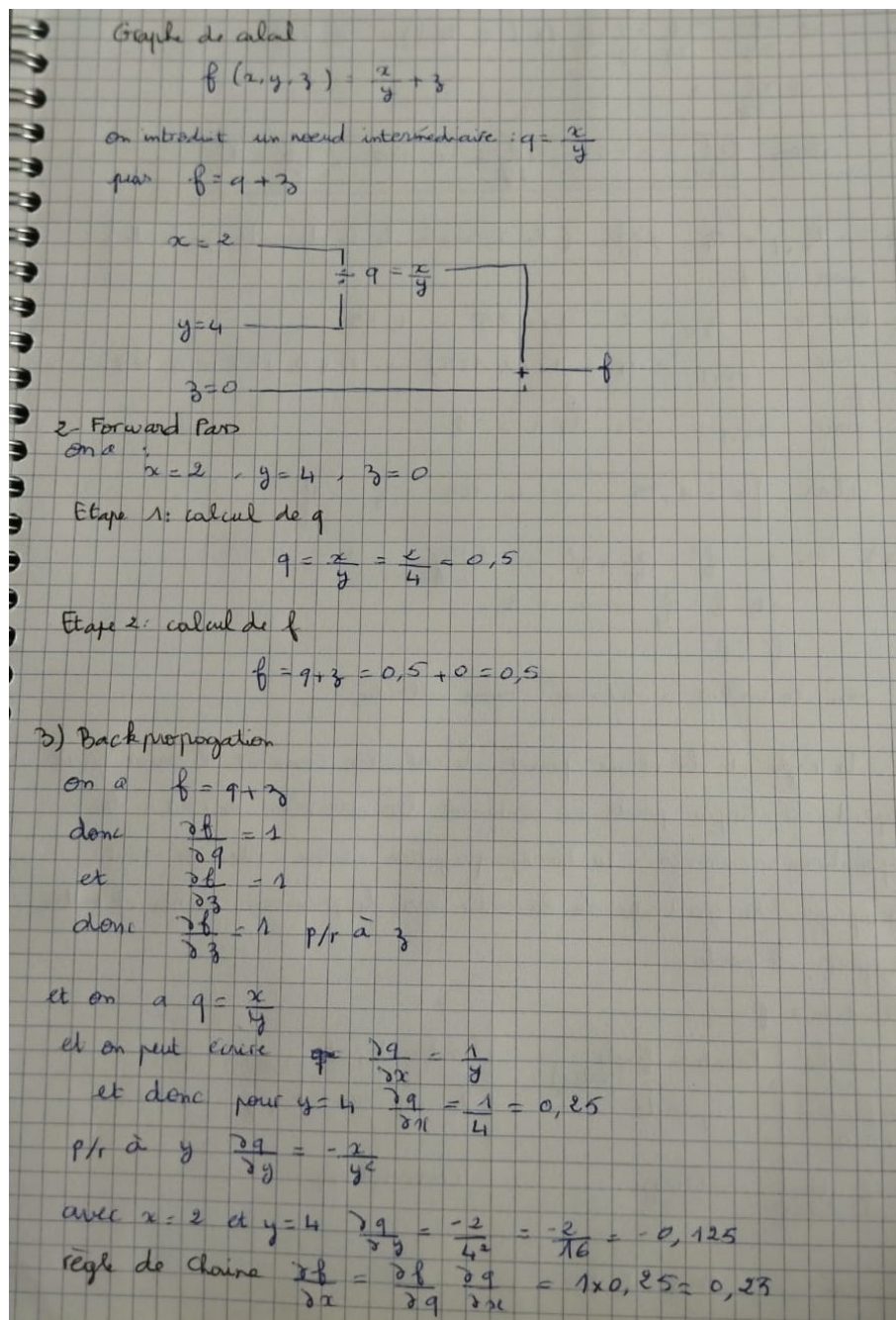


FIGURE 8 – Schéma du MLP (3-4-2) et nombre de paramètres, sans puis avec biais ET Dimensions du forward pass

3.2 Graphe de calcul et rétropropagation



Question. Quelles sont les principales raisons d'utiliser des mini-batches plutôt que d'optimiser sur un seul exemple à la fois ou sur l'ensemble total des données ?

Un seul exemple donne un gradient très bruité et instable, tandis que l'ensemble complet est trop coûteux à traiter à chaque mise à jour (mémoire, temps de calcul) et ne permet qu'une seule mise à jour par époque. Le mini-batch offre un compromis : un gradient suffisamment stable pour converger correctement, tout en profitant de la parallélisation GPU et en permettant plusieurs mises à jour par époque.

3.3 Mise à jour des poids ET Association sortie / fonction de perte

$\text{et } \frac{\partial f}{\partial y} = \frac{\partial f}{\partial y} \frac{\partial y}{\partial z} = 1 \times (-0,125) = -0,125$
 $\text{et } \frac{\partial f}{\partial z} = 1$
 alors $\frac{\partial f}{\partial x} = 0,25, \frac{\partial f}{\partial y} = -0,125, \frac{\partial f}{\partial z} = 1$

$\theta_{e+1} = \theta_e - \eta \frac{\partial f}{\partial \theta}$ (la formule de la descente gradient)
 avec $\eta = 1$
 Pour x : $x_{e+1} = x - 1 \times 0,25 = 1,75$
 pour y : $y_{e+1} = y - 1 \times (-0,125) = 4 + 0,125 = 4,125$
 pour z : $z_{e+1} = z - 1 \times 1 = -1$
 alors $x' = 1,75, y' = 4,125, z' = -1$

3) on a $f' = \frac{x'}{y'} + z' = \frac{1,75}{4,125} - 1 = -0,57$
 et $f = \frac{2}{4} + 0 = 0,5$
 $f' < f$
 oui la valeur a diminué comme attendu

la règle de la chaîne permet de calculer comment l'erreur final dépend de chaque poids en combinant les dérivées de toutes les opérations successives. la backpropagation utilise cette règle pour propager les gradients de la sortie vers les premières couches et mettre à jour les poids.

Question 3-f

tâche	Fonction finale (sortie)	Fonction de perte (loss)
classification binaire	sigmoïde	Binary Cross entropy
classification multi-classes	softmax	Cross entropy
Régression	ReLU identité	MSE

4 Premier réseau de neurones sur CIFAR-10

Question. Expliquez brièvement à quoi servent les arguments `batch_size` et `shuffle` dans le `DataLoader`. Pourquoi `shuffle` doit-il avoir une valeur différente pour l'entraînement et pour le test ?

- `batch_size` : nombre d'exemples traités ensemble à chaque itération (une mise à jour des poids par batch).
- `shuffle` : mélange l'ordre des exemples à chaque époque.
- À l'entraînement, `shuffle=True` évite que le modèle apprenne l'ordre des données et décorrèle les batchs successifs. Au test, il n'y a pas d'apprentissage : le mélange est inutile et un ordre fixe (`shuffle=False`) rend l'évaluation reproductible.

Question. Dans la méthode `forward`, pourquoi utilise-t-on `torch.flatten(x, 1)` avant de passer les données à la couche linéaire ?

Une image a la forme $(N, 3, 32, 32)$ alors qu'une couche linéaire attend un vecteur par

exemple, soit $(N, 3072)$. `flatten(x, 1)` aplatit tout sauf la dimension 0 : la dimension du batch est préservée.

Question. *Pourquoi est-il crucial de ne pas ajouter de fonction d'activation Softmax à la fin du réseau quand on utilise `nn.CrossEntropyLoss` ?*

`CrossEntropyLoss` intègre déjà un `LogSoftmax` suivi d'un `NLLLoss` et attend des logits bruts. Ajouter un `Softmax` l'appliquerait deux fois, ce qui fausse la loss et les gradients et dégrade la stabilité numérique.

Question. *Quelle est la différence fondamentale entre `optimizer.zero_grad()` et `loss.backward()` ?*

`loss.backward()` calcule les gradients de la loss par rapport aux paramètres (rétropropagation) et les accumule dans `.grad`. `optimizer.zero_grad()` remet ces gradients à zéro pour que ceux du batch précédent ne s'ajoutent pas à ceux du batch courant.

Question. *Pourquoi utilise-t-on `with torch.no_grad()` : lors de l'évaluation ? Quel est l'avantage en termes de ressources matérielles ?*

En évaluation on ne rétropropage pas : PyTorch n'a pas besoin de construire le graphe de calcul ni de stocker les activations intermédiaires. On économise de la mémoire GPU et du temps de calcul.

Question. *Si votre classificateur prédisait les classes de manière purement aléatoire, à quelle précision devriez-vous vous attendre sur le test CIFAR-10 ?*

CIFAR-10 compte 10 classes équilibrées : environ $1/10 = 10\%$.

5 Utilisation de TensorBoard

Question. *Pourquoi est-il important d'inclure la date, l'heure et les hyperparamètres dans le nom du dossier de logs (`run_name`) ?*

Chaque exécution obtient un dossier unique : aucun run n'écrase le précédent, on sait avec quels hyperparamètres une courbe a été obtenue, et TensorBoard peut superposer et comparer plusieurs expériences (traçabilité et reproductibilité).

Question. *Dans l'onglet Scalars, à quel niveau de smoothing distinguez-vous clairement la tendance sans masquer des changements importants ? Pourquoi observe-t-on autant de bruit sur `Loss/train_step` comparativement à `Loss/train` ?*

Niveau de smoothing retenu : 0.68

`Loss/train_step` est calculée sur un seul mini-batch de 32 exemples : elle a une forte variance qui dépend des exemples tirés. `Loss/train` est une moyenne sur les 45 000 exemples de l'époque, ce qui lisse ces fluctuations.

Question. *Analysez les courbes `Loss/train` et `Loss/val` superposées pour les 3 runs. Lequel donne la meilleure accuracy en validation ?*

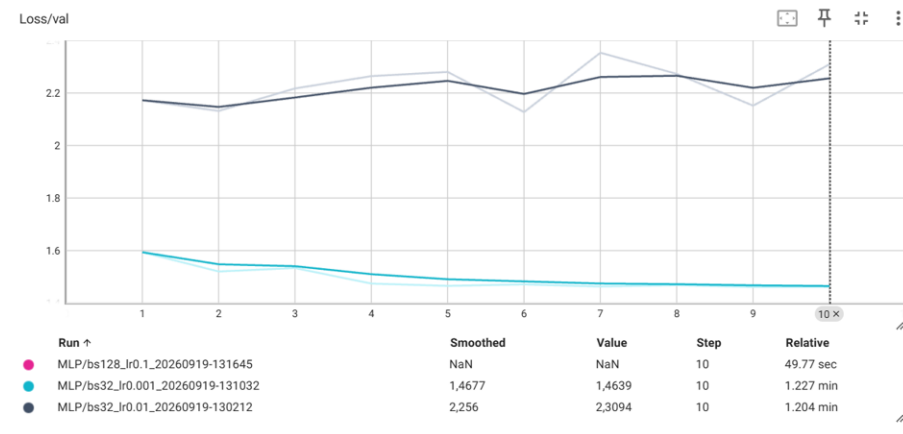


FIGURE 9 – Loss/train des trois runs.

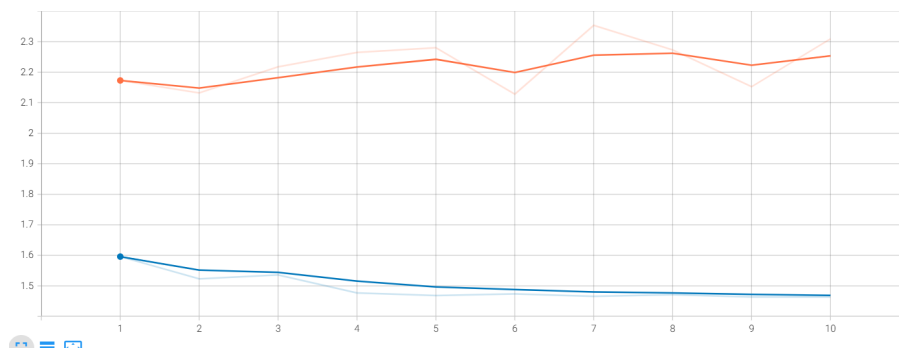


FIGURE 10 – Loss/val des trois runs.

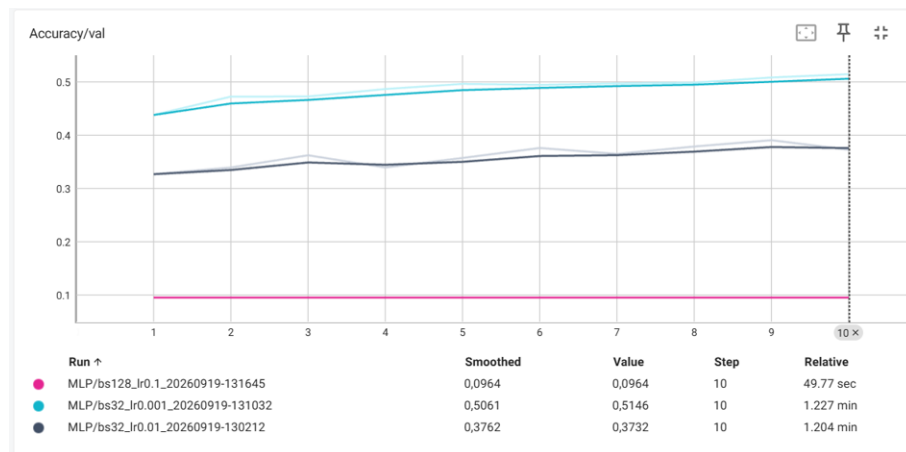


FIGURE 11 – Accuracy/val des trois runs.

Run	Learning rate	Batch size	Val accuracy finale
1	10^{-2}	32	0,373
2	10^{-3}	32	0,515
3	10^{-1}	128	0,096

Meilleur run : Run 2, le meilleur : sa loss de validation baisse de 1,60 à environ 1,46 puis

se stabilise, alors que sa loss d'entraînement continue de descendre jusqu'à 1,11. C'est le seul qui montre un début de sur-apprentissage. Run 1 : le learning rate est trop grand, la loss d'entraînement baisse à peine (2,08 puis 1,93) et la loss de validation reste instable. Il n'a pas convergé. Run 3 : il diverge (la loss devient NaN) et son accuracy de 0,096 est au niveau du hasard.

Question. *Comment détecte-t-on visuellement un sur-apprentissage sur les courbes de perte d'entraînement et de validation ? Décrivez l'allure des courbes.*

La loss d'entraînement continue de diminuer, alors que la loss de validation atteint un minimum puis remonte (ou stagne). L'écart entre les deux courbes se creuse au fil des époques : le modèle mémorise le jeu d'entraînement et généralise moins bien.